Page No.
Date

Function in C++

Topics to be covered on this Sections ⇒

1. Introduction
2. Function components
3. passing data to functions
4. Function Return data Type
5. Library Functions
6. Parameter passing
7. Return by Reference
8. Default Arguments
9. Inline Function
10. Function overloading
11. Function Templates / Generic programming
12. Arrays and Function
13. C++ Stack
14. Scope and extent of variables
15. Storage classes
16. Function with Variable number of arguments
17. Recursive Function
18. Complete Syntax of main().

Introduction:-

⇒ It is difficult to implement large program even if its algorithms is available.

⇒ To implement such program with ease, it should be split into a number of independent tasks, &

⇒ That tasks can be easily designed, implemented and managed.

⇒ These types of tasks is called functions.

⇒ A repeated group of instructions in a program can be organised as a function

The advantages of functions include the following :-

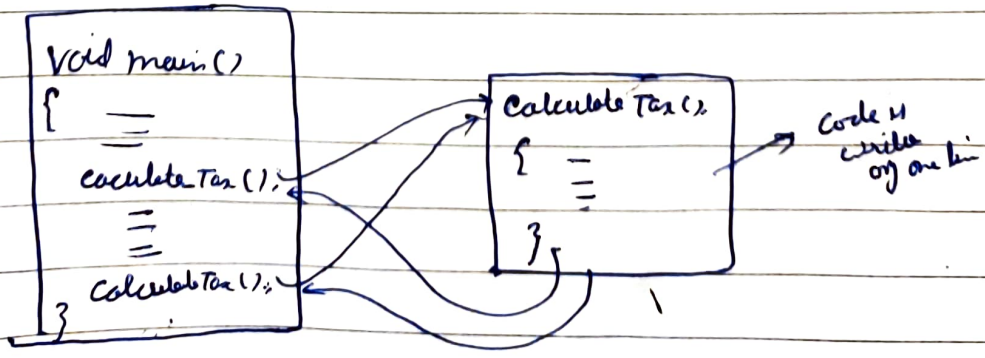
(a) Reduction in the amount of work and development time

(b) Debugging of program becomes easier.

(c) Reduction in size of program

(d) Function can be easily accessed without redevelopment.

(e)



Page No. _____
Date

⇒ Function components ⇒

* Every function has the following elements associated with it
  - ⇒ Function declaration or prototype
  - ⇒ Function parameters (Formal parameters)
  - ⇒ Combination of function declaration and its definition
  - ⇒ Function definition
  - ⇒ Return statement
  - ⇒ Function call

```
Void func(int a, int b) → function prototype/declaration
```

Formal parameter

```
Void func(int a, int b)
{
  - - -
  - - - -
  - - - -
}
- - - - -
- - - -
func(x,y);
- - - -
- - -
```

actual parameter.

```
#include <iostream>
using namespace std;
int max(int x, int y);
Void main()
{ int a, b, c;
  cout << "Enter two integers";
  cin >> a >> b;
  c = max(a,b)
  cout << "max(a,b):" << c << "\n";
}
```

ⓐ Function prototype
ⓑ Function definition
ⓒ Function call
ⓓ Function parameters
ⓔ Function Return
ⓕ Elimination of function prototype

'\xCD'

Page No.
Date

Passing Data to Function:

- Passing Constant as arguments
  Ex → void percentage chart (int percent)
- Passing variables as arguments.
  Ex → Enter percent of Raj, Shri. Cin >> m1, m2; Pur[m4];
- Passing multiple Argument

```
# include <iostream>
using namespace std;
Void percentageChart ( int percentage , Char Style );
int Void main ( );
{ int m1, m2, m3, m4;
  Cout<<" Enter the precentage of Shri, Raj, wages, Sunil";
  Cin >> m1 >>m2>> m3>> m4 ;
  Cout<<" Shri :";
  cout << percentageChart (m1, '*');
  Cout <<"Raj:";
  Cout <<" percentageChart (m2, '\xCD');
  Cout <<"Wages:";
  Cout <<" PercentageChart (m3, '~');
  Cout <<" Sunil:";
  Cout << PercentageChart(m4,'!');
  return 0;
}

Void prentageChart ( int percentage, Char Style )
{  for( int i =0, i < Percentage , i++ )
   { cout << style;
     cout << endl;
} }
```

Page No. _____
Date

* Function Return Data Type :-

The return value can be a constant, a variable, a user-defined data struct. or a pointer to a function.

* Generic/~~Programming~~ Template in C++

Inline Function

Function Overloading

~~Function~~ parameter passing →
- Call by value
- Call by Refere.
- Call by pointer

Return by Reference

~~Default~~ Arguements

* Templates in C++

Templates provide a mechanism for creating a single function possessing the capabilities of several functions which only differ in their parameter

```
#include <iostream>
using namespace std;
void swap(int *x, int &y)
{ int temp;
    temp = *x;
    *x = *y;
    *y = temp;
}

void swap(char *x, char *y)
{ char temp;
    temp = *x;
    *x = *y;
    *y = temp;
}

int main()
{ int a, b;
cout<<"enter numbers";
cin>>a>>b;
 swap(&a, &b);
cout<<"no. after swapping"<<a<<b;
  char ch1, ch2;
cout<<"Enter characters";
cin>>ch1>>ch2;
  swap(&ch1, ch2);
cout<<"no. after swapping"<<ch1<<ch2
```

With the help of template

```
template <class T>
void swap (T *x, T *y)
{ T t;
  t = x;
  *x = *y;
  *y = t;
}
```

* Inline function ⇒

```
#include <iostream>
using namespace std;
int funmul (int a, int b)
{ return a*b;
}
int main ()
{ int a, b;
  cout << "Enter the number" << "\n";
  cin >> a >> b;
  cout << "The multiplication of two number:" << funmul(a,b);
```

Page No.
Date

Function overloading

```
#include<iostream>
using namespace std;

int Sqr_area(int x);
int Rect_area(int x, int y);
int Cir_area(float x);

int main()
{
  int edge_length;
  cout<<" enter le
```

Templates in C++
function overload
Inline Function
Call by refere
Return by Refer
Defaul funct →

```
#include

void printLine(char = '-', int n=70);
int main()
{
  printLine();
  printLine('*');
  printLine(30);
```

Page No.
Date

Return by Reference -

```
int & max(int &x, int &y);
void int main()
  { int a, b;
    cout<<" enter the
      cin>>a>>b;
    max(a, b)=425;
    cout<<a<<b;
  }
int & max(int &x, int &y)
  {
    if (x>y)
    return x
```

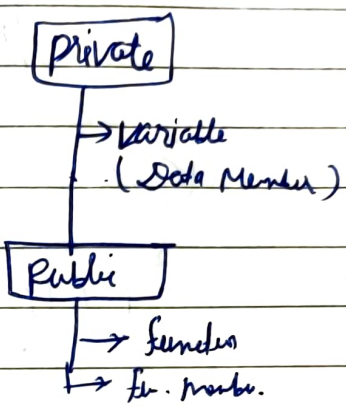
Page No. _____
Date

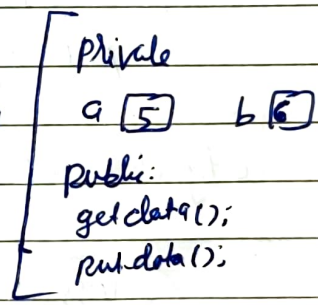
* Class and object → Access Specifier
* Array of objects
* object as a parameter / argument

```
#include <iostream>
using namespace std;

Class addition
{ private:
    int a, b;
  public:
  | void getdata();
    void putdata();
};
int main()
{ addition aa;
   aa.getdata();
   aa.putdata();
} return 0;

void addition::getdata()
{ cout<<"Enter two numbers"<<endl;
   cin>>a>>b;
}
void addition::putdata()
{ cout<<"Addition of two no"<<a+b;
}
```

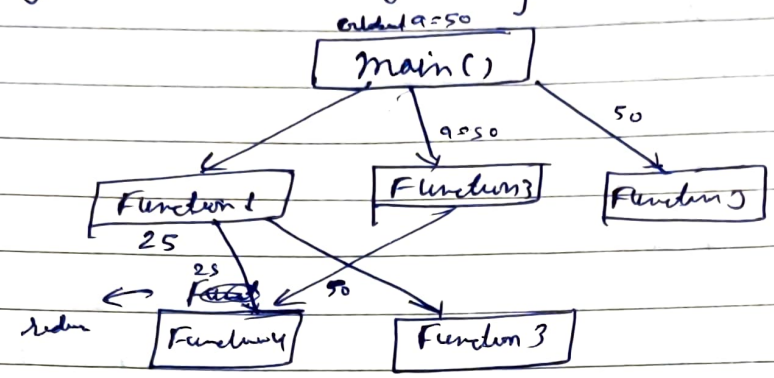





```
int main()
{ addition aa[5];
   for(int i=0, i<5; i++
   { aa[i].getdata();
   }
}
```

Page No.
Date

* Object-Oriented Programming →



C is → POP (procedural oriented prog.)

C++ → OOP (Object oriented programming)

Class → It is the collection of data members and functions put together into a single unit.

Object → It is the instance of class.

Class → Object

↳ it is representation of class

① Universe → class

Milky-way Galaxy → Object

② Milky-way Galaxy → class

Earth → Object

③ Earth → class

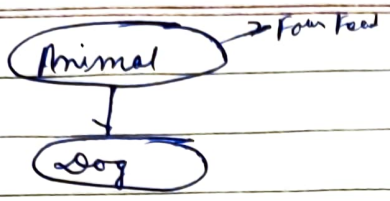
India → Object

④ India → class

Delhi → Obj

⑤ Delhi → class

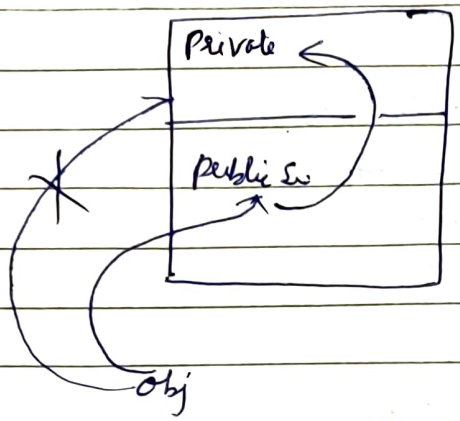
Rohtak → obj



1 hr, 50 min
2 hr, 55 min

50 (0) 105 (1
55 60
105 45

* Private, public, protected



protected → Access specifier

It is just like private

T_1	hr	min
T_2	hr	min
T_3	hr	min

Return by Refer

```
int max(int &a, int &b);
int main()
{  int x, y;
   cout << "En a ..
   cin >> x, y)
   max(x, y) = 425;
   cout << "n - -
}
int & max(int &a, int & b)
  { if (a>y)
```

re.

* Pointer Inside a class
Passing object as a parameter
Returning objects from function.

* Write a program in C++ that add marks of two Student.

```
#include<iostream>
using namespace std;
class Student
{
  int marks;
  int totalmarks=0;
  public:
    void getMarks()
    { cout<<"Input marks of Students";
      cin>>marks;
    }
   void printMarks()
   { cout<<"Total marks of the Student"<<totalmarks;
   }
   Student addmarks(Student S2)
   {
     Student S;
       S.totalmarks = marks + S2.marks
       return S.
}   ;
int main()
{ Student S1, S2, S3;
   S1.getmarks();
   S2.getmarks();
   S3 = S1.addmarks(S2)
}      S3.printMarks()
```



* Write a program in C++ that add two Times

```
#include<iostream>
using namespace std;

Class TimeAddition {
  int hrs, mins;
  public:
   Void getData()
   { cout << "Enter hrs and mins";
     cin << hrs << mins;
   }
   void printData()
   {
     cout << "Addition of two times is : " << hrs << "hours" << mins
          << "Minute";
   }
   Void addTime(TimeAddition t1, TimeAddition t2)
   { hrs = (t1.mins + t2.mins)/60;
     mins = (t1.min + t2.min) %.60
     hrs = hrs + t1.hrs + t2.hrs;
} };
int main()
{ TimeAddition t1, t2, t3;
  t1.getData();
  t2.getData();
  t3.addTime(t1, t2);
```

	hrs	mins
t_1	5	55
t_2	6	25
t_3	12	20

Page No.
Date

* Friend Function =>

```
# include <iostream>
Using namespace std;

class complex {
    int a, b;
    public:
        void setnumber (int n1, int n2) {
            a = n1;
            b = n2;
        }
        void printnumber () {

            cout << " your number is " << a << "+" << b << "i";
        }
    friend complex Sumcomplex (complex O1, complex O2);
};
int main() {
    complex C1, C2;
    C1
complex SumComplex (complex O1, complex O2) {
    complex O3;
    O3.SetNumber ((O1.a+O2.a), (O1.b+O2.b))
}
int main () {
 complex C1, C2;
 C1.Setnumber (5, 6);
 C2.Setnumber (8, 6);
 Sum = Sumcomplex (C1, C2);
 Sum.printnumber ();
} return 0;
```

Page No.
Date

Properties
⇒ Not in the scope of the class
⇒ Since it is not in the scope of the class, it can not be called from the object of that class
⇒ Can be invoked without the help of any object
⇒ Usually contains the argument of the object.
⇒ Can be declared inside public or private section of the class.
⇒ It can not access the members directly by their names and need object name. member name to access any member.

```
* #include <iostream>
using namespace std;
// Forward declaration
class Complex;
   class Calculator {
      public:
         int add(int a, int b){
            return (a+b);
         }
         int SumRealComplex(Complex, Complex);
   }

class Complex{
   int a, b;
   friend int Calculator :: SumRealComplex(Complex, Complex);
   public:
      void Setnumber(int n1, int n2){
         a=n1; , b=n2;
      }

int Calculator :: SumRealComplex(Complex O1, Complex O2){
      return (O1.a + O2.a);
   }
```

friend class Calculator;

```
int main () {
Complex O1, O2;
O1.Setnumber (1,4);
O2.Setnumber (5,7);
Calculator Calc;
int res = Calc.SumRealComplex(O1, O2);
Cout<<" The Sum of real part of O1 and O2 is "<<res<<endl;
return 0;
}
```

---

* WAP to add two complex number using friend fn.

```
#include <iostream>
using namespace std;

Class complex {
int a, b;
public:
  Void Setnumber (int x, int y) {
      a = x;
      b = y;
  }

  Void printnumber. (int a, int b) {
  Cout<<" The compx.
  }          addComplex
  friend Complex (Complex C1, Complex C2) {
  { Complex C3;
        C3.printn ((C1.a + C2.a), (C1.b + C.
    ret C3;
  }
```

Page No. _____
Date

* Constructor -

```
class integer {
  int m, n;
  public:
    integer();
};
integer :: integer()
{ m=0; n=0;
}
int main(){
  integer int;
  return 0;
}
```

* parameterized constructor

```
class integer {
  int m, n;
  public:
    integer(int a, int b);
};
integer :: integer(int a, int b)
{ m=a; n=b; }
int main{
  integer int1(0,100);
  integer int1 = integer(0,100);
  rdu 0;
}
```

* Multiple constructor / Constructor overload

* Copy constructor -

Page No.
Date

* Constructors
Parameterized Constructors
Deconstructor
Constructor overloading
Constructors with default arguments
Nameless objects
Dynamic initialization through constructor
Constructor with dynamic operations
Copy Constructor
Constructors for two dimentional arrays
Constant objects and constructor
Static datamembers with constructor and destructors.
Nested classes.

Constructor overloady -

```
#include<iostream>
using namespace std;
class Account{
    int accno;
    float balance;
    public:
        Account()
        { cout<<"Enter the account no. for acct object:";
          cin>>accno;
          Cout<<"Enter the balance";
          Cin>>balance;
        }

    Account(int acc)
    { accno=acc;
      balance=0.0;
```

```
Account ( int acc, float bal )
{ accno = acc;
  balance = bal;
}
void display ()
{
  cout << "Account no is:" << accno << endl;
  cout << "Balance is: " << balance << endl;
}
Void moneyTransfer ( Account &acc, float amount);
};

Void Account :: moneyTransfer ( Accout &acc, float amount)

{   balance = balance - amount;
    acc.balance = acc.balance + amount;
}

int main ( ) {
  Account acc1;
  Account acc2 ( 10001);
  Account acc3 ( 10002, 20000);
  Count<<"Account Information: <<endl,
  acc1.display ();
  acc2.display ();
  acc.display (); → int transmoney;
  cout<<" How much money you want to tranfer acc1 to acc
  Cin>> transMoney,
  acc3.money transfer ( acc1, 200);
  cout<<" updated account information '<<endl,
  acc1.display ();
```

* Constructors with default argument =

```
#include <iostream>

class complex {
    float real;
    float imag;
  public:
    complex()
    { real = imag = 0.0; }

    complex( float real_part , float imag_part = 0.0);
    { real = real_part;
      imag = image_part;
    }
    void show( char *msg ).
    { cout << msg << real;
      if (img < 0)
        cout << "-i";
      else
        cout << "+i";
      cout << img << endl.
    }
  complex add( complex c2);
};

complex complex:: add ( complex c2 ){
  complex temp;
    temp.real = real + c2.real;
    temp.img = imag + c2.img;
    return temp;
```

```
int main()
{ Complex C1(1.5, 6.5);
  Complex C2(2.5);
  Complex C3  C1.Show("C1=");
  Complex     C2.Show("C2=");
Complex C3 = C1.add(C2);
 C3.Show("C3=");
 return 0;
}
```

* **Deconstructor →**

```
#include<iostream>
using namespace std;
class Test
{ public:
    Test();
   ~Test();
};
Test::Test()
{ cout<<"Constructor of the class called"<<endl;
}
Test::~Test()
{ cout<<"Deconstructor of the class Test called"<<endl;
}
int main()
{ Test test1;
  return 0;
}
```

```
#include(iost..
class Test{
 public:
  Test();
 ~Test();
}
Test::Test(){
{
Test::~Tes{
~
```

When an object is no longer needed, it can be destroyed. A class have another special function called the destructor, which invokes automatically when an object is destroyed.

```
# include <iostream>
  int no_object = 0;
     int no_object_alive = 0;
class myclass{
      public:
         myclass()
        { ++no_object;
          ++no_object_alive;
        }
        ~myclass()
        {--no_object_alive;
        }
       void show()
      {: cout<<"Total number of objects created:"<<no_objects;
        cout<<"No. of objects currently alive"<<no_object_alive;
       }
};
void main(){
  myclass obj1;
  obj1.show();
  {
    myclass obj1, obj2;
    obj1.show(); // obj2.show();
  }
 obj1.show();

 myclass obj2; obj2,
   obj2.show(); // obj2.show();
 return 0;
}
```

Page No.
Date

* Nameless objects -

```
#include <iostream>
class nameless {
  int a;
    public:
     nameless () {
       cout << "Constructor" << endl;
     }
     ~ nameless () {
       cout << "Destructor" << endl;
       }
};
int main ()
{ nameless (); //nameless object is created as well as destroyed here
  nameless n1;
  nameless n2;
  cout << " program Terminates" << endl;
  return 0;
}
```

```
int main ()
  { myclass obj1;
    obj1.show ();
    { myclass obj1, obj2;
      obj1.sh ()
    }
    myc obj2, cm
    ~,
```

```
class nameless {
  public
   nameless () {
   cout << "cm
   ~ nameless () {
       cout << "
   }
};
int main () {
  nameless ();
  ~ ~;
```

* Dynamic Initialization of object using Constructor =

```
#include <iostream>
using namespace std;
class name {
  private:
    char first[15], second[15], last[15];
  public:
    name()                                  // Constructor 1
    { first[0] = second[0] = last[0] = '\0'; }
    name(char * FirstName);                 // Constructor 2
      strcpy(first, FirstName);
    name(char *FirstName, char * SecondName);
    name(char *FirstName, char *SecondName, char *LastName);
    void display(char *msg);
};
name::name(char *FirstName) {
    strcpy(first, FirstName);
    second[0] = last[0] = '\0';
}

name::name(char *FirstName, char *SecondName) {
    strcpy(first, FirstName);
    strcpy(second, SecondName);
    last[0] = '\0';
}
```

```
Void Show( char *msg )
{ cout << msg << endl;
  cout << "First Name" << first;
  if ( middle[0] )
   cout << "mid Name" << mn
  if (last[0]).
    cout << "la.
}
int main()
 { name obj1, obj2, obj3; // Constructor 1
   obj1 = name ("waqar"); //.
   obj2 = name ("waqar", "ahmad"); //
   obj3 = name ("waqar", "ahmad", "jamil");
   obj1.Show("First Person detail");
   ob ..
}
```

```
main ()
{ Employee;
```

* Copy constructor =

```
#include <iostream>
using namespace std;
class Wall {
  private
    double length;
    double height;
  public:
    Wall ( double len , double hgt )
    { length = len;
      height = hgt;
    }
```

```
Wall ( Wall &Obj) {
  length = Obj.length;
  height = Obj.height;
}
double calculateArea() {
  return length * height;
}
int main() {
  Wall (5.0, 10.8);
  Wall Wall1 = Wall;
  // Wall Wall1(Wall);
cout << "Area of Wall 1:" << Wall1.calculateArea() << endl;
cout << "Area of Wall 2:" << wall2.calculateArea();
return 0;
}
```

Page No.
Date

Static data members with constructor and destructor.

```
#include <iostream>
using namespace std;
class Employee {
    int id;
    static int count;
public:
    void setData();
    { cout << "Enter the id" << endl;
      cin >> id;
      count++;
    }
    static void getCount(void) {
        // cout << id; // throw error
        cout << "The value of count is:" << count << endl;
    }
};
int Employee :: count; // Default value is 0

int main() {
    Employee Aman, Sahil, Waqar;
    Aman.setData();
    Aman.getData();
    Employee :: getCount();

    Sahil.setData();
    Sahil.getData();
    Employee :: getCount();

    Waqar.setData();
    Waqar.getData();
    Employee :: getCount();
    return 0;
}
```

```
void getData()
{ cout << "The id of this employee is:" << id << "and this is employee number" << count; }
```

90% → {7 days → week, 3

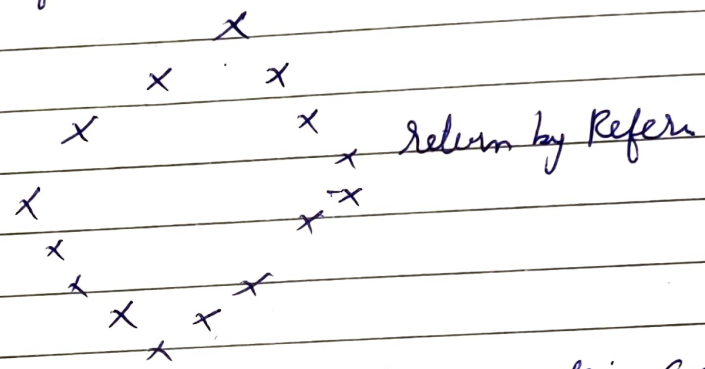
Page No.
Date

Object, Class, Se

* Lab Programs →

① WAP to implement workflow applical using Default const Inline

② Calculate distance b/w two point using Friend Function default con
~~Return by reference~~



③ WAp to multiply two mxn and nxp matrix and calculate the number of multiplication performed in the implementation
using data members
i. State of const static function
ii. Copy constructor
iii. Object as a argument.
iv. pointer ~~as~~ to Object.

Pointer to Object =

```
#include <iostream>
using namespace std;

class Box {
  private:
double length;
double width;
double height;
  public:
    Box(double l, double w, double h) {
      length = l;
      width = w;
      height = h;
    }
    double Volume() {
      return length * width * height;
    }
int main() {
  Box Box1(25.0, 20.0, 30.0);
  Box Box2(20.0, 10.0, 5.0);
  Box *ptr; // ~~Box *ptr = new Box;~~
  ptr = &Box1;                    (*ptr).Volume().
  cout << "Volume of Box1" << ptr->Volume() << endl;
  ptr = &Box2;
  cout << "Volume of Box2" << ptr->Volume() << endl;
  return 0;
}
```

~~Self referential class~~ -

* Revising pointer using new and delete operator.

```
#include <iostream>
using namespace std;

int main() {
    int a;
    int* ptr = &a;
    cout << " The value of a is " << *(ptr) << endl;

// By using new keyword/operator
    int *p = new int(5);
    cout << " The value at address p is " << *(p) << endl;

    float *p = new float(5.3);
    cout << " The value at address p is " << *(p) << endl.

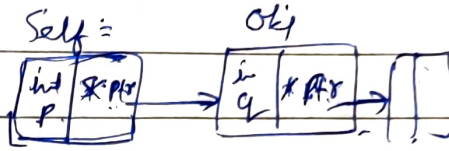
    int *arr = new int[3];
      arr[0] = 10;
    *(arr+1) = 30;
      arr[2] = 40;
    cout << " The value of arr[0] is " << arr[0] << endl;
    cout << " The value of arr[1] is " << arr[1] << endl.
    cout << " The value of arr[2] is " << arr[2] << endl;

// By using delete keyword/operator.
    delete arr;
    delete [] arr;
    return 0;
}
```

* Pointer to Object
* Array of objects
* Self-referential classes.

⇒ Self referential classes are those classes in which one or more pointers points to the class of the same type.

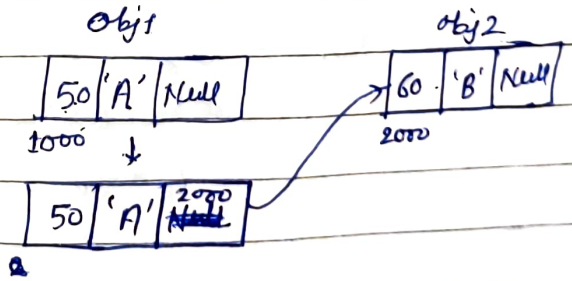
```
class Self {
    int p;
    Self *ptr;
}
```




```
# include <iostream>
using namespace std;

class Code {
public:
    int a;
    char b;
    Code *ptr;
};

int main()
{ Code obj1;
    obj1.a = 50;
    obj2.b = 'A';
    obj1.ptr = NULL;
  code obj2;
    obj2.a = 60;
    obj2.b = 'B';
    obj2.ptr = NULL
    obj1.ptr = &obj2
    cout << obj1.ptr->a << obj1.ptr->b;
  return 0;
```



```
#include<iostream>
using namespace std;

class ShopItem
{ int id;
  float price;
  public:
    void setData(int a, float b){
      id=a;
      price=b;
    }

    void getData(){
    cout<<"Code of this item is"<<id<<endl;
    cout<<"price of this item is"<<price<<endl;
    }
};
int main(){
  int size=3;
  ShopItem *ptr = new ShopItem[size];
  ShopItem *ptrTemp = ptr;
  int p, i;
  float q;
  for(i=0;i<size;i++)
  { cout<<"Enter Id and price of item"<<i+1<<endl;
    cin>>p>>q;
    ptr->setData(p,q);
    ptr++;
  }
  for(i=0;i<size;i++)
  { cout<<"Item number"<<i+1<<endl;
    ptrTemp->getData();
    ptrTemp++;
```

* **Access specifier in Inheritance ⇒**

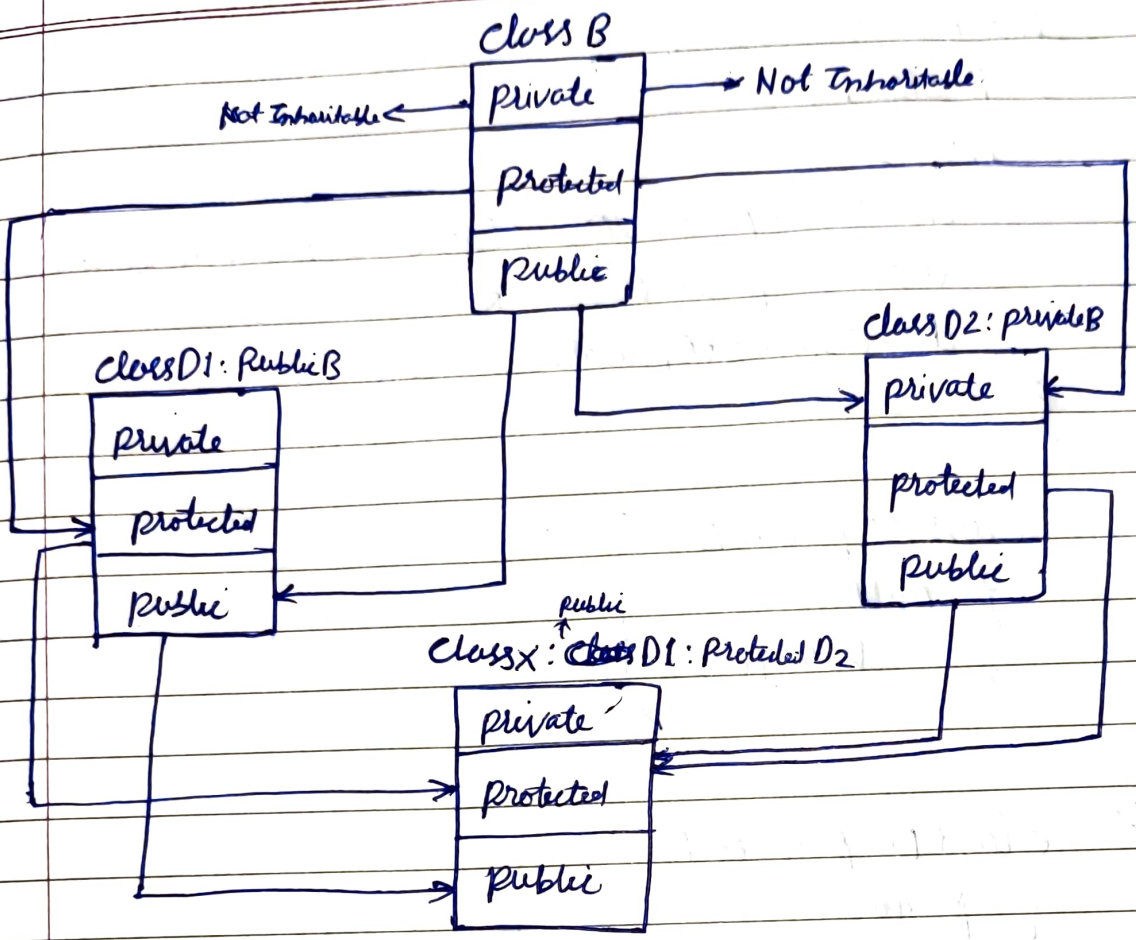
```
class name {
    private :      // visible to member function within
    ---- --              its class but not derived class
    ------

    protected :    // visible to member function within its
    ---- --              class and derived class
    ------

    public :       // visible to member func. within its class
    ---- --              derived classes and through object.
};
```

Important Topics of Inheritance :

⇒ Introduction
⇒ Class Revised
⇒ Derived class Declaration
⇒ Forms of Inheritance
⇒ Inheritance and member accessibility
⇒ Constructors in derived class



* Example ⇒ (Public Inheritance)

```
class B
{ private:
    int privateB;
  protected:
    int protectedB;
  public:
    int publicB;
    int getBprivate()
    { return privateB;
    }
};
```

```
class D : public B
{ private:
    int privateD;
  protected:
    int protectedD;
  public:
    int publicD;
    void myfunc()
    { int a;
      a = privateB; x
      a = getBPrivate(); ✓
      a = protectedB; ✓
      a = publicB;
    }
};
```

```
int main()
{ D objd;
  int d;
     d = objd.protectedB; //Error: 'B::protectedB' is not accessible
     d = objd.publicB ; // ok
     d = objd.getBprivate(); //OK
}

class x : public D
{ public:
      void g();
}

* class D: private B
{ private:
      int privateD;
  protected:
     int protectedD;
  public:
     int publicD;
     void myfunc()
     { int a;
       a = privateB ; //Error
       a= getBprivate();
       a = protectedB;
       a = publicB;
     }
int main() {
  obj objd;
  int d;
     d = objd.protectedB // Error.
     d = objd.publicB // Error.
```

```
d=objd.getprivate(); // Error.
}
```

* Constructors in derived classes:

→ In the application of inheritance, objects of the derived class are usually created instead of the base class

→ Hence, it makes sense for the derived class to have a constructor and pass the arguments to the constructor of the base class

→ When an object of a derived class is created, the constructor of the base class is executed first and later, the constructor of derived class

⇒ NO constructors in the base class and derived class

```
#include <iostream>
using namespace std;
class B {
        // body of base class without constructor
};
class D: public B { // body of derived class without const
public:
    void msg() {
    cout<<"No constructor in base and derived class"<<endl
    };

int main()
{ D objd;
  objd.msg()
  return 0;
}
```

* Multilevel Inheritance:

```
#include<iostream>
Const int MAXLEN=25;
class person{
 private:
   char name[MAXLEN];
   Char gender;
   int age;
 public:
   Void ReadData(){
    Cout<<"Name?"; Cin>>name;
    Cout<<"gender?"; Cin>>gender;
    Cout<<"age?"; Cin>>age;
   }
   void DisplayData(){
    Cout<<"Name:"<<name<<endl;
    Cout<<"Gender"<<gender<<endl;
    Cout<<"age:"<<age<<endl;
   }
};

class student: public person
 { private:
     int rollno;
     char branch[20];
   public:
     void ReadData(){
       person::ReadData();
       Cout<<"Rollno"<<endl; Cin>>rollno;
       Cout<<"Branch:"<<endl; Cin>>branch;
     }
```

```
    void DisplayData() {
      person::DisplayData();
      cout<<"Roll No. "<<rollno;
      cout<<"Branch "<<branch;
    }
};

class exam:: public Student {
  protected:
    int Submarks1;
    int Submarks2;
  Public:
    void ReadData() {
      Student::ReadData();
      cout<<"Enter Subject 1 marks:"; cin>>Submarks1;
      cout<<"Enter Subject 2 marks"; cin>>Submarks2;
    }

    void DisplayData() {
      Student::DisplayData();
      Cout<<"Marks scored in Subject 1 "<<Submarks1;
      cout<<"Marks scored in Subject 2 "<<Submarks2;
    }

    int totalmarks()
    { return Submarks1 + Submarks2;
    }
};

int main() {
    exam annual;
    annual.ReadData();
    annual.DisplayData();
```